Campaign Planner

AI-Powered Workflow System · Airtribe GenAI Builder core project

Track 3, Marketing & Growth: Campaign Generator · Henil Shah

Deliverable	Link
Live app (Claude page)	https://claude.ai/artifact/5GBzAsTthBhjmvgjoZrrCg
GitHub	https://github.com/henil-dev/campaign-planner
Demo video	*add before submitting*

Tools. Built in Claude Code, versioned on GitHub, published as a Claude page (a claude.ai artifact) with Claude as the model. The Airtribe team approved Claude in place of Replit, whose publishing needs a paid plan. The same code also runs as a Node/Express server with Gemini for local development (section 2).

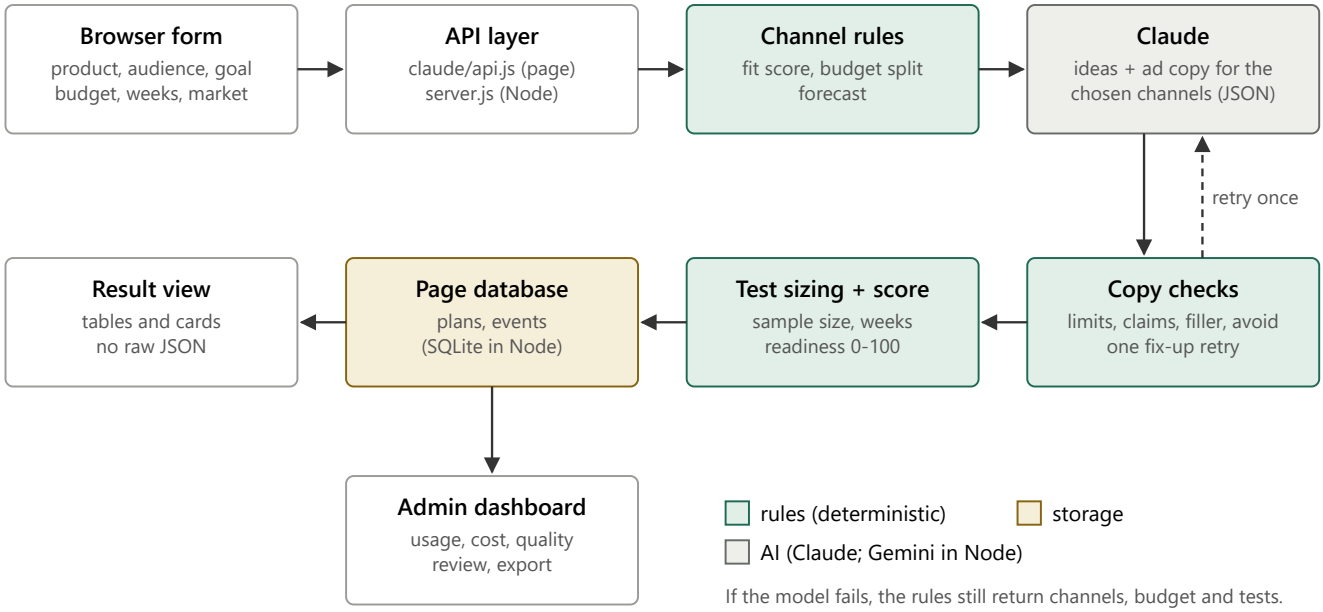
1. What it does and why it is built this way

A founder or growth marketer enters a product, who it is for, a goal and a monthly budget. The app returns a campaign plan: which channels to use and how much to spend on each, three campaign ideas, two ad versions per channel, an A/B test plan, and a readiness score that lists every problem found.

The design choice that shapes everything else: **the model writes, the rules decide and check.** A language model is good at angles, hooks and ad copy. It is bad at arithmetic and at saying "your budget can't support this". Ask a chatbot for a plan on ₹40,000 a month and it will suggest five channels and three A/B tests, none of which the budget can feed. So:

- Channel choice, budget split, forecasts and test sizing are plain functions with fixed numbers. The same input always gives the same answer, and every number can be traced.
- Claude only writes for the channels the rules already picked, in a fixed JSON shape.
- The copy is checked by rules after generation. Hard failures go back to the model once.
- If the model fails, the user still gets the rule-based half of the plan.

2. System architecture



Request flow. Rules run on both sides of the AI call.

Two ways to run, one pipeline. `lib/planner.js` takes the model as a plug-in, and every page talks to the backend through one small interface (`window.API`). That gives two builds from the same files:

	Live Claude page (submitted)	Local Node server
Where the pipeline runs	In the viewer's browser	In Express (`server.js`)
Model	Claude, through the page's `sample` capability, on the viewer's own Claude account	Gemini 2.5 Flash over REST
Storage	The page's database (`db` capability): `plans` and `events` collections	SQLite: `plans` and `events` tables
Backend interface	`claude/api.js`	`public/api.js` calling the REST API
Admin access	Only people who can edit the page	HTTP Basic auth via `ADMIN_PASSWORD`
Pages	One page with `#new`, `#plans`, `#plan-…`, `#admin` views	`/`, `/plans`, `/plan?id=N`, `/admin`

Layer	Files	Responsibility
Frontend	`public/index.html`, `form.js`, `render.js`, `views.js`, `admin.js`; `claude/index.html` + `claude/main.js` for the Claude page	Structured form, client checks, renders every part of the plan as tables and cards
Backend interface	`claude/api.js` (Claude page), `server.js` + `public/api.js` (Node)	Validation, saving, feedback, admin actions; Node adds a rate limit of 20 plans/hour per IP
Logic layer (before AI)	`lib/logic/channelPlan.js`, `lib/channels.js`	Channel fit scores, channel selection, budget split, forecast

Layer	Files	Responsibility
AI	`lib/prompt.js`; adapters in `claude/api.js` and `lib/llm.js`	Prompt with the chosen channels and budgets plus a JSON schema; Claude adapter on the page, Gemini adapter (timeout, retry, model fallback) in Node
Logic layer (after AI)	`lib/logic/copyCheck.js`, `testPlan.js`, `readiness.js`	Copy checks and one fix-up retry, A/B test sizing, readiness score
Orchestration	`lib/planner.js`	Runs the steps in order and handles the failure cases
Storage	page database (`claude/api.js`), SQLite (`lib/db.js`)	Plans and events
Monitoring	`lib/dashboard.js`, admin view	Usage, AI success rate, cost, quality breakdowns, review workflow, from the same function in both builds

Request flow for one plan (`API.createPlan`):

1. `validateInput` cleans and checks every field. Failures come back with a message per field, and a `validation_failed` event is logged.
2. `planChannels` scores the six channels, picks the ones that fit and that the budget can fund, splits the money and forecasts results.
3. `buildUserPrompt` lists the chosen channels with their budgets and ad formats. The model adapter sends it with the JSON schema (Claude on the page, Gemini in Node).
4. `shapeOutput` keeps only known fields and only the chosen channels, and trims lengths.
5. `checkCopy` runs the copy rules. If there is any hard error, or two or more filler/generic hits, the issues are sent back in a second turn. The version with fewer problems is kept.
6. `sizeTests` works out sample size and time for each test against the channel's forecast traffic.
7. `readiness` scores the plan from 100 down.
8. The plan is saved: summary fields for the dashboard plus the full input and result, and events are logged.
9. The browser renders the result. No raw JSON is ever shown.

3. Frontend

Plain HTML, CSS and JavaScript with no framework and no bundler. `npm run build:claude` assembles the Claude page from the same form and dashboard markup the Node pages use, so the two never drift apart. Colours are theme tokens with a light and a dark palette, so the page follows the viewer's claude.ai theme.

Input form (`/`). Product name, business type, a description, the audience, one goal (reach, leads, sales or installs), monthly budget, length (2 to 12 weeks) and market. Optional: average order value, tone, and anything to avoid. Three "try an example" links fill realistic inputs (a clay cookware brand, a B2B invoicing tool, a JEE coaching batch). Every field is checked in the browser first, with the message under the field. `validateInput` repeats every check before the pipeline runs.

Result view. The form is swapped for the plan. Sections are numbered, and each one is labelled **rules** or **AI** so it is clear which part came from where:

Section	Source	What the user sees
Summary strip	rules	Readiness score and level, channels, estimated results per month and cost each, ROAS or cost per customer

Section	Source	What the user sees
Channels and budget	rules	Each channel with its fit score and reasons, budget and share bar, clicks, results, cost each. Channels not used and why
Positioning, ideas	AI	One-line positioning, audience insight, three idea cards
Ad copy	AI, checked by rules	Two versions per channel with character counters against the channel limit, a copy button, and the issues found on each version
A/B test plan	AI tests, sized by rules	Hypothesis, A and B, metric and baseline, sample per version, time needed, verdict
Readiness score	rules	Every deduction with its points
Check before launch	AI	Assumptions the model made that the user should confirm

Saved plans (`#plans`, `#plan-<id>`). Searchable list. A saved plan renders from the stored JSON exactly as it was first shown and asks "Was this plan useful?". If the viewer is signed out or only has view access, the plan is still shown with a note that it was not saved.

The layout works down to phone width (tables scroll inside their box, the page itself never scrolls sideways).

4. Backend and the logic layer

4.1 Backend interface

Both builds expose the same methods on `window.API`: `createPlan`, `listPlans`, `getPlan`, `feedback`, `health`, and for admins `stats`, `plans` (filtered), `setReview`, `remove` and `exportCsv`. On the Claude page they read and write the page database directly; CSV export goes through the page's download prompt. On the Node server they map to these routes:

Method	Path	Purpose
POST	`/api/plan`	Validate, run pipeline, save, return the plan with its id
GET	`/api/plans`, `/api/plans/:id`	List and fetch saved plans
POST	`/api/plans/:id/feedback`	Useful / not useful
GET	`/api/admin/stats`	Dashboard numbers (takes the browser's time zone offset)
GET, PATCH, DELETE	`/api/admin/plans[/:id]`	Filtered list, set review status, delete
GET	`/api/admin/export.csv`	CSV export
GET	`/api/health`	Server up, AI key present, model name

4.2 Channel fit and selection

Six channels: Google Search, Instagram & Facebook, YouTube, LinkedIn, creators/influencers, Reddit & Quora. Each gets a score from 0 to 100:

> score = 45% × business-type fit + 35% × goal fit + 20% × audience signal

Business-type fit and goal fit come from hand-set 0–5 tables in `lib/channels.js` (for example LinkedIn is 5/5 for B2B software and 0/5 for a D2C brand). The audience signal is a set of word patterns: "owners", "CA firms", "managers" point to LinkedIn; "students", "parents", "JEE" to YouTube; "Instagram", "home cooks" to Instagram and creators; "chase", "by hand today", "looking for" to Google Search.

Selection rules:

- Channels scoring under 45 are dropped as a weak fit.
- The number of channels is capped by budget: 1 under ₹30k, 2 under ₹75k, 3 under ₹2 lakh, 4 above.
- Each channel has a minimum monthly spend to get out of the ad platform's learning phase (₹10k for Reddit/Quora up to ₹40k for LinkedIn, higher for international markets). A channel is only added if the budget still covers every chosen channel's minimum.
- If nothing fits, the channel with the best fit per rupee of minimum spend is used, and the plan says the budget is below the usual minimum.

4.3 Budget split and forecast

Budget is split in proportion to score^2, so the strongest fit gets a clear lead. Any channel left below its minimum is topped up from the others, then amounts are rounded to ₹500.

For each channel: cost per click = base CPC × category multiplier (B2B 1.8×, finance 2×, local 0.85×, …) × market multiplier (India nationwide 0.8×, international 5×). Clicks = budget ÷ CPC. Results = clicks × goal conversion rate × channel intent (search 1.6×, YouTube 0.55×). Cost per result, ROAS (sales goal with an order value) and cost per customer (leads goal, assuming 1 in 10 leads buys) follow. All forecasts are shown as a ±30% range. If the cost per sale or per customer is above the order value, the plan warns.

4.4 Copy checks and the retry

Check	Level	Example
Length over a hard platform limit	error	Google headline over 30 characters, description over 90
Length over where the platform truncates	warning	Instagram primary text over 125 characters, YouTube script over 55 words
Risky claims	error in finance, health and education; warning elsewhere	"guaranteed", "100%", "#1", "cures", "risk-free", "assured returns", "top rank"
Filler wording	warning	"unlock", "seamless", "elevate", "game-changer", "look no further"
Goes against the user's "avoid" note	error	Mentions a discount when the user said no discounts; uses a word after "no …"
Generic	warning	Mentions no price, place, name or other detail from the input
A and B too similar	warning	Over 70% word overlap, so the test would teach nothing
Missing channel	error	The model skipped a chosen channel

If the first draft has any error, or two or more filler/generic hits, the issue list goes back to the model as a second turn ("fix only these problems, keep everything else"). Both drafts are checked and the one with fewer issues is kept. Anything left is shown on the ad.

4.5 A/B test sizing

Each test is sized for a two-sided test at 95% confidence and 80% power, to detect a 20% relative change:

> n per version = $(1.96 + 0.84)^2 \times [p_1(1-p_1) + p_2(1-p_2)] \div (p_2 - p_1)^2$, with $p_2 = 1.2 \times p_1$

The baseline p_1 is the channel's click-through rate (for headline and creative tests) or its conversion rate (for offer and landing-page tests). Traffic per week comes from the channel's forecast, split 50/50. Tests on the same channel

run one after another. If a test can't finish within the campaign, the plan says so and works out the smallest change that would show up with the traffic available.

4.6 Readiness score

Starts at 100. Points come off: 50 if there is no AI copy, 20 if the budget is under the channel minimum, 10 if cost per sale or customer is above its value, 8 per copy error (max 40), 3 per copy warning (max 15), 10 per test that can't get enough traffic (max 20). 80 and above is "Ready to launch", 60–79 "Review flagged items", below 60 "Needs rework".

4.7 Worked example: Kulhad Co.

Clay cookware, D2C, goal online sales, ₹60,000 a month for 4 weeks, metro India, order value ₹1,800. The audience mentions Instagram.

Channel	Business fit	Goal fit	Signal	Score	Result
Instagram & Facebook	5	5	yes	100	₹32,000
Creators / influencers	5	4	yes	93	₹28,000
Google Search	3	5	no	62	not used: ₹60k only stretches to 2 channels
YouTube	3	2	no	41	not used: weak fit

Instagram at ₹9 a click buys about 3,550 clicks; at 1.44% conversion that is about 51 sales (₹625 each), for a ROAS near 2.9. Across both channels: about 70 sales at ₹858 each, ROAS 2.1. A headline test on Instagram needs about 38,800 impressions per version and finishes in a week. An offer test on creators needs about 35,500 clicks per version, far more than ₹28,000 buys, so the plan marks it "not enough traffic". That is exactly the kind of advice a chat answer leaves out.

4.8 Failure handling

Situation	What happens
Bad or missing input	A message under each field; nothing is generated
Viewer signed out of claude.ai, or declined to let the page use Claude	Rule-based plan and a default headline test; a banner says the AI step was skipped and why
Claude usage limit, refusal, empty or cut-off answer	Rule-based plan with a plain message for that case
Model returns text around the JSON	The outermost JSON object is extracted; if none parses, it counts as an AI error
Copy breaks rules	One fix-up turn; leftovers shown as tags on the ad and counted in the score
Viewer has view-only access	Plan shown in full with a note that it could not be saved
Node only: Gemini timeout (60s), 429 or 5xx	One retry after 1.5s, then a rule-based plan with the error message
Node only: too many requests	20 plans per hour per IP, then HTTP 429

5. Database design

The Claude page stores plans in its built-in document database, in two collections. The Node build keeps the same fields in SQLite (`better-sqlite3`, WAL mode, `data/campaigns.db`), with `input_json` / `result_json` columns in place of nested objects.

`plans`: one document per generated plan.

Column	Type	Notes
id	document id	time-based id, used in `#plan-<id>` (auto-increment integer in SQLite)
created_at	ISO timestamp	UTC; plans are listed newest first
product_name, business_type, goal, market	TEXT	from the input, for search and filters
budget, weeks	INTEGER	
status	TEXT	`complete`, `ai_error`, `no_key`
readiness_score, readiness_level	INTEGER, TEXT	
channels	list	channel keys chosen by the rules
error_count, warn_count, tests_blocked, retried	INTEGER	quality counters for the dashboard
model, latency_ms, error		AI call metadata for speed and failures (Node also records tokens for cost)
issues	list	each copy issue's text and level, for the dashboard breakdown
review_status	TEXT	`new`, `approved`, `archived`; set by an admin
feedback	INTEGER	1 useful, −1 not useful, NULL no answer
input, result	objects	the full input and the full plan, so it re-renders exactly

`events`: an append-only log (`plan_saved`, `retry`, `ai_error`, `validation_failed`, `review_status`, `plan_deleted`, and `rate_limited` in Node) with an optional `plan_id`.

Why summary fields and the full plan in one document: the dashboard needs counts and filters over small fields, and a saved plan has to re-render exactly as first shown, which the stored result gives without a dozen child records. Plans are readable by people the page is shared with; writing needs at least Contributor access, and review status and delete need edit access.

6. Admin dashboard

The admin view is shown only to people who can edit the Claude page (others see a short note instead, and the nav link stays hidden). In the Node build it is `/admin`, behind HTTP Basic auth when `ADMIN_PASSWORD` is set.

- **Health line**: whether Claude and the page database are connected in this view.
- **Usage and quality**: plans (and last 7 days), AI success rate, average readiness, share of drafts sent back for fixes, average AI time, and AI cost. On the Claude page, calls run on each viewer's own Claude plan, so the page shows no per-token bill; the Node build prices tokens at Gemini list rates. A note line adds feedback, approvals, plans with tests the budget can't support, and rejected form submissions.
- **Charts** (plain CSS, no chart library): plans per day for 14 days with AI failures stacked in red, readiness mix, channels picked by the rules, copy problems caught by type (must-fix vs warning).
- **Plans table**: search and filter by readiness, AI status and review state; open a plan, set it to approved or archived, or delete it.
- **Event log**: the last 40 events with links to plans.
- **CSV export** of all plans, through the page's download prompt.

7. Git commit progression

One commit per stage, each one leaving the app runnable.

#	Commit	Date	Message
1	`ae84a9e`	18 Oct 2026, 00:43	Set up Express project, Replit config and README with the build plan
2	`246b889`	18 Oct 2026, 00:45	Add campaign input form with examples and client-side checks
3	`acb68c6`	18 Oct 2026, 00:48	Add /api/plan: server validation, Gemini call with JSON schema, structured result view
4	`3a2eebc`	18 Oct 2026, 00:53	Add logic layer: channel fit scoring, budget split, forecasts, copy checks with one retry, test sizing and readiness score
5	`c2e8176`	18 Oct 2026, 00:56	Store plans and events in SQLite; add saved plans list, plan page and feedback
6	`42ff363`	18 Oct 2026, 00:58	Add admin dashboard: usage, AI cost, readiness and issue breakdowns, plan review, CSV export, event log
7	`a1bc9a9`	18 Oct 2026, 01:02	Add README, documentation and scripts to build the PDF and screenshots
8	`bf18774`	22 Oct 2026, 22:33	Make the pipeline model-agnostic and route the pages through one API layer

8. Limits and what I'd do next

- Forecast rates are planning benchmarks kept in one file, not live auction data. Next: let users enter their own past CPC and conversion rate and use those instead.
- The "avoid" check covers discounts and simple "no X" phrases. A second, cheap model call could judge free-text instructions.
- The generic-copy check is a proxy (does the copy mention anything from the input?). Feedback data from the admin page could tune it.
- Plans are shared with everyone who can open the page. For real teams, each person's plans would move under a private per-user path in the page database.